

內容摘要

這篇論文提出了 Skylight，是一套結合「軟體微基準 + 硬體觀測」的方法，用來**逆向工程 drive-managed SMR 硬碟的內部行為**。作者先在 Linux 上以 fio 撰寫一系列 I/O 微基準，量測精細的延遲變化，再搭配在硬碟上開窗、用高速攝影機追蹤磁頭位置，從而推論出 SMR 硬碟內部 STL (Shingle Translation Layer) 的設計細節。

在方法部分，作者先實作了三種「基於文獻中的 STL 演算法」的 SMR 模擬器(兩個 set-associative STL、一個 fully-associative STL)，以 Linux device mapper 疊在一顆 CMR 硬碟上，確認 Skylight 測試流程能否準確還原已知參數，接著再把同一套測試套件跑在兩顆實體 Seagate SMR 硬碟(分別是 5TB 與 8TB)上，以量測結果與磁頭軌跡來推回實際產品的 STL 設計。

透過這些測試，作者成功推論出多個關鍵設計：例如 Seagate SMR 採用單一位於外圈之 persistent disk cache，可用容量約 20GiB (5TB) 與 25GiB (8TB)，並以 journal entries 在 persistent cache 上連續記錄更新資料；同時資料帶(band)大小約 15–40MiB，使用 static LBA→PBA 對應、採類似 FIFO 的清理順序的 aggressive cleaning。

論文最後整理出這樣的 drive-managed SMR 對 workload 的影響：關閉 volatile cache 時，順序與隨機寫入延遲都明顯高於 CMR；啟用 volatile cache 或提高 queue depth 時，隨機寫入可以非常快，但 cache 填滿後效能會急遽下降，需要長時間 idle 讓背景清理追上。作者據此建議 **SMR 比較適合桌機式、低 duty-cycle、有長 idle 的 workload**，而不一定適合高負載伺服器情境。

設計細節和模擬評估

1. Skylight 方法設計與 SMR 模擬環境

I. 軟體部分：fio 微基準與一系列「測試腳本」

作者利用 fio 寫出多種針對性 I/O pattern，例如：小區域內隨機寫、交錯寫(寫一塊跳一塊)、長時間 idle 後再讀、遞增寫入大小與 queue depth

等，觀察平均延遲、抖動與 IOPS 的變化。作者一共設計了 14 個 microbenchmarks，以下只挑其中幾組重點測試說明：

- TEST 1：以固定區域隨機寫，觀察寫入延遲是否固定，以區分 SMR vs CMR，並辨別 persistent cache 是 disk 還是 flash。
- TEST 2, 3：透過 fragmented reads 與 idle 之後的延遲變化，推得 persistent cache 的位置（外圈 / 內圈 / 中圈、多個 cache 與否）和清理風格（lazy vs aggressive）。
- TEST 6–8：控制寫入大小與 queue depth，利用「cache 填滿開始清理」的臨界點，反推出 persistent cache raw size、map entries 上限，以及 host 可見的實際 cache size。
- TEST 11–12：藉由被清理 band 的 read latency pattern，估計 band 大小與 block mapping 是 static 還是 dynamic。

II. 硬體部分：在硬碟上開窗 + 高速攝影機追蹤磁頭

為了驗證純延遲分析可能產生的多解，作者直接把 SMR 硬碟外殼鑽洞換成透明窗，使用 1,000 fps 相機拍磁頭位置，並以影像處理將每一 frame 對應到半徑位置。

- 例如在 TEST 1 中，隨機寫的延遲固定約 25ms，但只有透過磁頭軌跡才能確認：磁頭其實一直停在 OD、每第 240 次寫入會短暫 seek 到中間，代表在做 map merge。
- 在 fragmented read 測試中，攝影機畫面顯示磁頭反覆在 OD 與某個 LBA 區間之間來回，可以看出 persistent cache 的位置與清理時 RMW 走訪的流程。

III. SMR 模擬環境：三種 STL 實作

作者利用 Linux device-mapper 在一顆 CMR 硬碟上實作：

- Emulated-SMR-1：set-associative STL、persistent cache 37.2GiB、band size 40MiB、使用 disk 當 cache。
- Emulated-SMR-2：同為 set-associative，但 persistent cache 放在 SSD 上，模擬「flash cache + disk band」。

- Emulated-SMR-3: 類 fully-associative STL, 將 disk 切成 zones, 每個 zone 有 free bands, 可動態重新映射 band, 清理是 aggressive。這些模擬器都可以掛上檔案系統、甚至拿來編譯 Linux kernel, 代表其行為足以支撐真實 workload, 用來驗證 Skylight 測試方法的正確性。

2. 逆向工程結果與關鍵觀察

A. Drive 類型、cache 型態與清理策略

- 透過 TEST 1, Seagate 5TB SMR 在固定區域隨機寫時, 延遲幾乎固定在 $\approx 25\text{ms}$, 顯示寫入其實被 sequential log 到某一 persistent cache, 證實它是 SMR 而非 CMR。
- 測得在 queue depth=31、多種寫入大小時, 延遲會以 0.5 軌為單位跳動, 推得 drive 使用「2.5 軌的最小 internal write + journal entry 以半軌為粒度成長」的機制, cache 空間中有部分用於 metadata / map。
- TEST 3, 4 顯示 Seagate SMR 採 **aggressive cleaning**: idle 約 1 秒左右就會開始把 persistent cache 的更新搬回 band, 清理單一 band 的時間約 0.6–1.6 秒, 但在極端 adversarial pattern 下, 一個 band 可能要 45 秒才清完。

B. Persistent cache 容量、map 限制與 circular log 行為

- 利用小寫入 + queue depth=1 的情境, 觀察到清理在約 22,800 次寫後開始, 再配合假設的 track 容量, 推估 raw cache size 約 100GiB; 但因 journal entry 和 metadata overhead, host 實際能用的 cache 約 20GiB (5TB 版), 而 map entries 約 200k。
- 透過固定 10,000 個 LBA 重複寫的測試 (TEST 9–10), 作者證明 persistent cache 是以 circular log 形式管理: 當 log 頭繞回時, 若發現寫入位置對應的舊 entry 已過時就可以直接覆蓋, 否則就必須觸發清理。這也解釋了「重複寫少量 LBA」時, 清理延遲明顯比「廣泛散佈 LBA」來得低。

C. Band 大小、映射方式與 torn write 問題

- TEST 11 透過「先在大區域內每隔固定距離寫一點小資料, 再觀察清理後的 sequential read latency」, 推得 Seagate SMR 的 band 大小在不同

半徑區段之間會變化，大約落在 17–36MiB(5TB 型號)，8TB 則是 15–40MiB。

- TEST 12 證明 Seagate SMR 的 non-cached data 採 **static mapping**：清理後同一 band 的 track-to-track switch latency pattern 不會改變，與 fully-associative STL 的模擬結果不同。
- 在 static mapping 下，若開啟 volatile cache 且為了追求高 sequential throughput 而「直接覆寫 band」，一旦中途掉電就可能造成整個 band 內多個 track 一起損毀，類似 CMR 的 torn write 但粒度變成「好幾 MiB 的大區塊」。作者甚至用 Arduino 控制電源，實驗重現了硬碟在這種情境下產生 read error 的現象。

D. Zone 結構與 shingling 方向

- TEST 14 以 sequential 1MiB read 掃過整顆 disk，畫出 throughput 隨時間變化曲線，發現圖形呈現對稱、階梯式下降的 pattern，代表磁頭會在同一面讀完大量 tracks 才切換到下一面，形成「水平 zones」，大小約 4–20GiB。
- 搭配磁頭位移影像，作者推得：在 OD 與 ID 區域，shingling 方向都是往中間 (MD) 疊；而在中間區，layout 反而較接近傳統 serpentine。這樣的水平 zone 設計有利於分配等容量 bands、動態調整 band size，以及在 STL 內部做 static/dynamic mapping。

結論

Skylight 提出一套結合微基準與磁頭影像的黑箱逆向方法，幾乎完整重建商用 SMR 硬碟的 STL 設計，包括 persistent cache 容量與位置、band 大小、mapping 方式與清理策略，並以模擬器驗證測試流程有效性。作者進一步指出，drive-managed SMR 較適合低 duty-cycle、具長 idle 的桌機型工作負載，而不利高併發伺服器情境，為後續設計 SMR-aware 系統提供關鍵參考。